

Decision Trees and Pegasos SVMs from Scratch: Price and Price-Tier Prediction on bina.az Listings

Surname1, Name1 Surname2, Name2 Surname3, Name3

MLE-AI-201 Machine Learning, AI Academy — Cohort II

Abstract—We implement a CART decision tree (Gini, entropy and MSE criteria) and a soft-margin support-vector machine trained with the Pegasos sub-gradient method entirely in NumPy, and apply them to 57,721 cleaned sale listings scraped from bina.az, Azerbaijan’s largest real-estate portal. One dataset is framed as two tasks: regression of $\log(\text{price})$ (Task A) and classification of a *premium* vs. *standard* price tier split at the training median (Task B). Every hyperparameter is selected on a validation split; final models are refitted on train+validation and scored once on a held-out test set, with 5-fold cross-validation for uncertainty. Our tree reaches a test RMSE of 0.224 on $\log(\text{price})$ ($R^2 = 0.873$), against 0.224 for scikit-learn’s tree with matched settings; a node-by-node comparison leaves 0 unexplained differences, with the remaining ones due to exact gain ties and scikit-learn’s float32 cast of the features. Our averaged Pegasos SVM reaches test F1 0.915 (scikit-learn LinearSVC: 0.915), and a random-Fourier-feature extension reaches 0.910. Error analysis shows that misclassified tiers concentrate around the threshold, and that the regression tree under-predicts the most expensive listings. Code and one-command reproduction: github.com/eyyubovsamur240-afk/ML-final-project (tag v1.0-final).

Index Terms—decision tree, support-vector machine, Pegasos, random Fourier features, real-estate price prediction.

I. INTRODUCTION

Asking prices on listing portals are noisy, heavy-tailed and driven by location, size and condition. We use them to study two classic learners from the inside: we write both models ourselves, feed them a real, uncleaned scrape, and benchmark them honestly against scikit-learn. Our contributions are:

- a leakage-free pipeline for the bina.az dump: tolerant parsing of Azerbaijani, unit-laden fields, currency conversion, documented cleaning rules, and preprocessing fitted on training rows only;
- a vectorised CART tree whose split search evaluates every threshold of every feature with cumulative sums, verified to make the same split as scikit-learn at every node up to exact ties;
- a Pegasos SVM with mini-batches, projection, iterate averaging and a random-Fourier-feature (RFF) kernel extension, whose objective we compare with LIBLINEAR’s exact optimum;
- a protocol that never touches the test split during selection, CV uncertainty for all models, error analysis, and bonus ensembles, ridge regression and clustering built on our own code.

II. BACKGROUND

Decision trees. CART [2] grows a binary tree greedily: each node picks the feature/threshold pair that maximises the impurity decrease $\Delta I = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$, where I is the Gini index $1 - \sum_k p_k^2$, the entropy $-\sum_k p_k \log_2 p_k$ (information gain [3]), or, for regression, the node variance $\frac{1}{N} \sum_i (y_i - \bar{y})^2$. Growth stops on depth, node-size or minimum-gain rules; leaves predict the majority class (with class frequencies as probabilities) or the mean.

SVMs and Pegasos. The soft-margin SVM [4] minimises the regularised hinge loss. Pegasos [1] solves this

primal by stochastic sub-gradient descent with step $\eta_t = 1/(\lambda t)$ and reaches ϵ accuracy in $\tilde{O}(1/(\lambda \epsilon))$ steps, independent of n . Random Fourier features [5] map $\mathbf{x}$ to $\phi(\mathbf{x}) = \sqrt{2/D} \cos(W\mathbf{x} + \mathbf{c})$ with $W \sim \mathcal{N}(0, 2\gamma I)$, so that $\phi(\mathbf{x})^\top \phi(\mathbf{z}) \approx e^{-\gamma \|\mathbf{x} - \mathbf{z}\|^2}$ and a linear solver on ϕ approximates an RBF-kernel SVM.

III. DATA AND PREPROCESSING

A. Dataset and cleaning

We use the Kaggle *Bina.az sale project* dump [13] (file `house_sale.csv`, SHA-256 prefix `b6a4c67e0d3c`): 100,775 rows scraped between 2024-10-05 and 2024-11-19. Column names are Azerbaijani (*Sahə, Otaq sayı, Mərtəbə, ...*), most numbers arrive as strings with units (“145 m²”, “7 / 9”, “2 640 AZN/m²”), and every price is already in AZN. All cleaning lives in `src/data_prep.py`:

1. names are transliterated and mapped to English (*area, rooms, floor, land area, repair, mortgage, bill of sale, category*); where the dump holds two versions of a field (repair as “Təmirli”/blank and as *var/yoxdur*) the complete one is used;
2. tolerant parsers handle thousands separators, decimal commas, “floor / total floors”, land in *soț*/m²/ha and yes/no flags; prices are converted to AZN at fixed rates;
3. the scraper visited many listings several times, sometimes after a price change: the 100,775 rows hold only 64,454 distinct listing URLs, and we keep the latest scrape of each;
4. we study *residential* listings (new builds, old stock, houses/villas; 58,452 listings). Land plots (area in *soț*, no rooms or floors), commercial objects, offices and garages (torpaq 3,315, obyekt 2,526, ofis 101, qaraj 60) are different markets; the scope is one setting in `config.py`;

Table 1. Cleaning steps and rows remaining.

Step	Rows left	Removed
raw rows	100,775	
repeat scrapes of a listing removed (latest kept)	64,454	36,321
residential categories only	58,452	6,002
has a positive price	58,452	0
has a positive area	58,452	0
exact duplicates removed	58,110	342
re-posted listings removed	57,737	373
price in [5,000, 15,000,000] AZN	57,728	9
area in [10, 3000] m ²	57,724	4
price/m ² in [30, 25,000] (typo filter only)	57,721	3
final cleaned rows	57,721	0

- rows without a positive price or area are dropped, then exact duplicates and re-posts of the same flat under a new listing id (same price, area, rooms, floor and coordinates);
- fixed plausibility rules remove typos and unit mix-ups (Table 1); impossible feature values (floor above the building height, coordinates outside Azerbaijan) are set to missing and imputed later.

The rules are constants chosen from domain knowledge, not statistics estimated on the data, so applying them before the split cannot leak.

Leakage. We drop **unit_price**, **total_price** because they are arithmetic functions of the target (price per m², price restated); the cleaning code also drops any other column whose name contains “price”. We also drop identifiers and free-form addresses (**id_x**, **rel_url**, **img_url**, **id_y**, **estate_id**, **estate_rel_url_y**, **owner_name**, **owner_title**, **shop_name**, **shop_title**, **address**, **estate_details_id_x**, **estate_details_id_y**, **estate_rel_url**, **listing_key**) and promotion flags plus scrape bookkeeping (**scraped_at**, **city_when**, **day_x**, **hour_x**, **vip**, **featured**, **products_label**, **datetime_scrape_y**, **currency_y**, **updated**, **views**, **day_y**, **hour_y**), which describe the seller’s advertising or the scraper rather than the property. Price per m² is used once, only as a typo filter, and never as a feature.

B. Features, tasks and split

From the cleaned rows we build 24 numeric/binary features (area and log area, rooms, area per room, floor, total floors, floor ratio, top/ground-floor flags, land area, latitude, longitude, distance to the Baku centre, repair/mortgage/bill-of-sale flags and six keyword flags from the description, e.g. “metro” or “dəniz” (sea)), plus one-hot encodings of category, building type, city, administrative district (parsed from bina.az’s location tags) and the finer location label (metro station or settlement), plus one 0/1 column per location tag (metro station, settlement or landmark listed on the ad) that appears in at least 20 *training* rows, 292 columns in total. One-hot rather than ordinal encoding is used because none of these has a natural order. Missing numeric values are imputed with the *training* median and flagged by an indicator, and a category level gets its own column only if it has at least 20 *training* rows; rarer or unseen levels fall into an “other” column.

Task A predicts $\log(\text{price})$: the raw price is strongly right-skewed (skewness 6.60; 0.32 after the log, Fig. 1a–b), and

Table 2. Dataset summary after cleaning.

Property	Value
Rows (raw / cleaned)	100,775 / 57,721
Features after encoding	292
Train / Val / Test	40,405 / 8,658 / 8,658
Tier threshold (train median)	208,000 AZN
Premium share train / val / test	50.0% / 50.1% / 50.0%
Median price (train)	208,000 AZN
Skewness price / $\log(\text{price})$	6.60 / 0.32

an error in log space is a relative error. The tree-based regressors (our tree, scikit-learn’s, and the forest and boosting bonuses) are fit to $\log(\text{price}/\text{m}^2) = \log(\text{price}) - \log(\text{area})$ and add the listing’s own $\log(\text{area})$ back, so they still output $\log(\text{price})$ and are scored on it. A leaf then holds a price per m² that each flat’s exact area scales, instead of one price for every flat in the leaf. The offset is the row’s own feature, so nothing leaks. Task B labels a listing *premium* if its price exceeds the *training* median, 208,000 AZN. The split is 70/15/15 (40,405/8,658/8,658), stratified on deciles of $\log(\text{price})$ with a fixed seed, so all splits share the price distribution and the tier balance (50.0%, 50.1%, 50.0% premium). Stratifying only decides which rows go where; no statistic is computed from validation or test rows.

Exploration (training split only, Fig. 1 and the appendix). The median price is 208,000 AZN (1st–99th percentile 46,000–1,250,000), the median price per m² is 2,235 AZN, and log area correlates with log price at 0.72. The median price per m² varies several-fold between districts (Fig. 9), so location carries a large part of the signal. After cleaning the most frequent missing fields are **building_type** 100.0%, **land_area_sot** 83.6%, **mortgage** 65.8%, **floor** 16.4%, **total_floors** 16.4%.

IV. METHOD

A. Decision tree (from scratch)

DecisionTree (`src/decision_tree.py`) builds Node objects recursively and exposes a scikit-learn-style `fit/predict/predict_proba` API. *Split search*: for a node with m rows we sort all candidate features at once (one `argsort` of the $m \times d$ block) and evaluate *every* threshold with cumulative sums. For classification, running class counts give the Gini or entropy of both children at every cut. For regression, running $\sum y$ and $\sum y^2$ give the children’s sums of squared errors, computed after centring y at the node mean for numerical stability. Candidate thresholds are midpoints between consecutive *distinct* sorted values. A split is valid only if both children keep at least `min_samples_leaf` rows, and the best gain wins (ties go to the lowest feature index). One node therefore costs $O(dm \log m)$, almost all of it inside NumPy.

Stopping and regularisation. A node becomes a leaf if it is pure, reaches `max_depth`, has fewer than `min_samples_split` rows, has no valid cut, or if the weighted gain $\frac{N}{N} \Delta I$ is below `min_impurity_decrease`. This is the same weighted definition scikit-learn uses, so matched settings mean the same thing in both libraries. Leaves store class frequencies (giving `predict_proba`) or the mean target. Feature importance is the total weighted

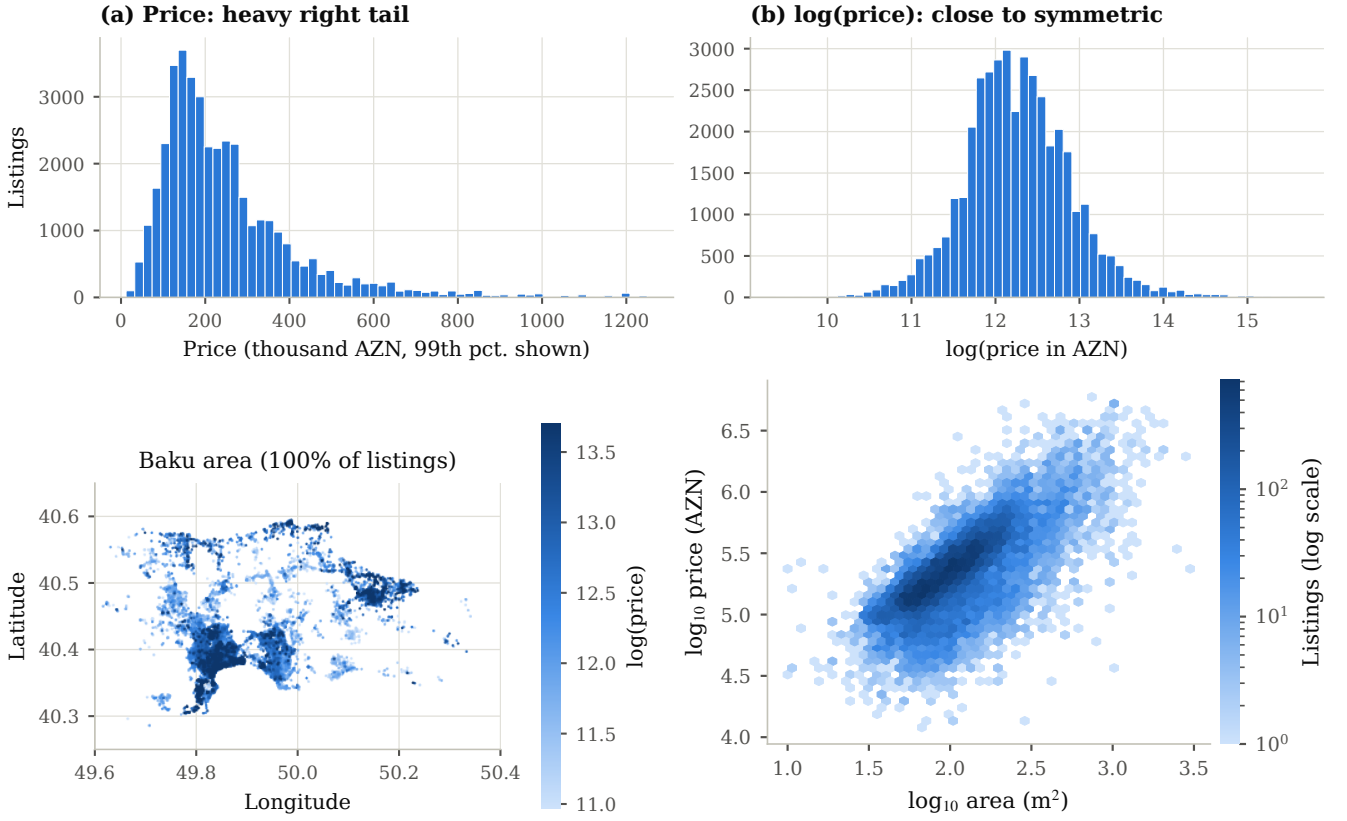


Figure 1. Exploration on the training split. (a, b) Price is heavy-tailed; $\log(\text{price})$ is close to symmetric. (c) Listing prices across Baku: the centre and the seafront are the most expensive. (d) $\log$ area vs. $\log$ price (correlation 0.72).

gain per feature, normalised to one (mean decrease in impurity). After fitting, the tree is flattened into arrays so that prediction routes all rows through the tree together.

Depth curves from a single fit. Greedy CART decides each node from that node’s rows only, so a tree grown to depth D and *truncated* at depth d is identical to a tree grown with $\text{max_depth}=d$. A unit test checks this equivalence, and we use it to draw depth curves at the cost of one fit.

B. Support-vector machine (from scratch)

PegasosSVM (src/svm.py) minimises

$$J(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \mathbf{w}^\top \mathbf{x}_i), \quad y_i \in \{-1, +1\}, \quad (1)$$

where $\mathbf{x}_i$ is augmented with a constant 1 so that the bias is the last weight. With a random mini-batch A_t of size k and the set A_t^+ of its hinge-active points ($y_i \mathbf{w}^\top \mathbf{x}_i < 1$), one step is

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda) \mathbf{w} + \frac{\eta_t}{k} \sum_{i \in A_t^+} y_i \mathbf{x}_i, \quad \eta_t = \frac{1}{\lambda t}, \quad (2)$$

followed by the optional projection onto the ball $\|\mathbf{w}\| \leq 1/\sqrt{\lambda}$, which contains the optimum. $k=1$ is classic Pegasos and $k=n$ is full-batch sub-gradient descent; the $\eta_0/\sqrt{t}$ and constant schedules are also implemented. Each epoch reshuffles the rows with a seeded generator and records $J(\mathbf{w})$ on the full training set. Because the last iterate is noisy when λ is small, the tuned solver returns the t -weighted iterate average $\bar{\mathbf{w}}_t = (1 - \frac{2}{t+1})\bar{\mathbf{w}}_{t-1} + \frac{2}{t+1}\mathbf{w}_t$, which keeps the optimal

$O(1/(\lambda T))$ rate [6]. Features are standardised with training statistics, since the hinge loss is scale-sensitive. We report $\|\mathbf{w}\|$, the margin width $2/\|\mathbf{w}\|$, and the support vectors, defined as the training points with $y_i f(\mathbf{x}_i) \leq 1$.

Non-linear extension: random Fourier features. We map the standardised features with $D=1,024$ random features and run the *same* linear solver on $\phi(\mathbf{x})$ (RFFPegasosSVM). We chose RFF over kernelised Pegasos because training and prediction cost $O(nD)$ instead of growing with the number of support vectors, which matters for $\sim 49,063$ training rows, and because it reuses the tested linear code unchanged. The cost is a Monte-Carlo kernel approximation, whose error we measure (Fig. 3, bottom-c).

V. EXPERIMENTAL SETUP

Protocol. (1) All hyperparameters are chosen on the validation split with models trained on the training split; the choice rule is fixed in code (best validation score, ties to the simpler model). (2) The chosen configurations are refitted on train+validation (49,063 rows) with imputation, encoding, scaling and the tier threshold re-learned on those rows (threshold 208,000 AZN), and scored once on the test split with 95% percentile-bootstrap intervals (1,000 resamples). Differences between two models are tested with a *paired* bootstrap on the same resampled test rows (Section D). (3) For uncertainty, 5-fold stratified CV on train+validation repeats the whole preprocessing inside every fold and reports mean $\pm$ std.

Metrics. Task A: RMSE, MAE and R^2 on $\log(\text{price})$, plus MAE (AZN) and MAPE after back-transforming with $\exp$,

which predicts the conditional median. Task B: accuracy, precision, recall and F1 of the premium class, ROC-AUC and PR-AUC (average precision) from continuous scores (leaf frequencies for trees, margins for SVMs), and confusion matrices. All metrics are our own NumPy code and are unit-tested against scikit-learn.

Baselines with matched settings. Our tree vs. DecisionTreeRegressor/Classifier with the same criterion, max_depth, min_samples_leaf and min_impurity_decrease. Our SVM vs. SGDClassifier (hinge, $\alpha = \lambda$, “optimal” schedule [7]) and LinearSVC ($C = 1/(n\lambda)$), LIBLINEAR [8], both behind a StandardScaler inside a Pipeline. Our RFF SVM vs. the exact-kernel SVC with the same γ and C . A constant predictor (training median) anchors Task A. All randomness is seeded (SEED=42).

VI. RESULTS

A. Tree: criteria, depth and the bias–variance trade-off

Fig. 2 shows the classic picture. Training error keeps falling with depth, while validation error bottoms out and then rises again as the tree starts fitting noise. An unpruned regression tree reaches a training RMSE of 0.057 at depth 25, but its validation RMSE is 0.249; the best unpruned depth is 14. Gini and entropy behave almost identically on Task B: their best validation F1 differ by 0.002 (Gini 0.923, entropy 0.921). Larger leaves regularise a fully grown tree, and min_impurity_decrease prunes it: at 10^{-2} the tree shrinks from 3,593 to 3 nodes (Fig. 11). On validation we select **gini, depth 14, min_samples_leaf 1** for Task B and **depth 14, min_samples_leaf 5, min_impurity_decrease 10^{-5}** for Task A (validation RMSE 0.222). In the learning curve (appendix) the train–validation gap goes from 0.114 to 0.036 as data grows, and validation error is still falling at the full training size, so more data would still help.

B. SVM: regularisation, margin and convergence

Table 3 and Fig. 3 (top) trace the effect of λ . A larger λ shrinks $\|\mathbf{w}\|$, which widens the margin $2/\|\mathbf{w}\|$ and makes more training points fall inside it as support vectors, trading training fit for a simpler boundary. Validation selects $\lambda = 10^{-4}$ ($C = 0.247$). At this setting the final model has $\|\mathbf{w}\| = 4.659$, a margin of 0.429, and 9,474 support vectors (19.3% of the training rows).

Fig. 3 (middle) shows the first 50 epochs, the budget each fit gets in the λ sweep. Small λ converges slowly, as the $1/(\lambda T)$ rate predicts, which is why averaging matters and why the chosen λ is refit for 500 epochs: over that run J falls from 1.000 at $\mathbf{w} = 0$ to 0.2020 and changes by 0.01% over the last five epochs. With the same epoch budget, mini-batches of 32 make as much progress per epoch as $k = 1$ at a fraction of the wall time (batch 1, last iterate: 30.09s; batch 32, last iterate: 3.74s; batch 32, averaged: 4.21s; full batch: 4.37s), while full-batch descent, which takes only one step per epoch, lags far behind. On the full training set, our averaged Pegasos ends at $J = 0.2053$, against LIBLINEAR’s exact optimum $J = 0.2045$ (gap 0.40%), and the two agree on 99.7% of test predictions.

For the RFF extension, validation selects $\gamma = 0.001$ and $\lambda = 10^{-5}$ (Fig. 3, bottom). Validation picks the smallest

Table 3. Linear Pegasos SVM vs. λ (train→validation; the selected row is bold). $C = 1/(n\lambda)$.

λ	C	$\ \mathbf{w}\ $	$2/\ \mathbf{w}\ $	SV	Tr F1	Val F1	Val AUC
10^{-6}	24.7	448.24	0.004	8.1%	0.920	0.908	0.969
3×10^{-6}	8.25	164.01	0.012	8.3%	0.920	0.909	0.969
10^{-5}	2.47	53.34	0.037	8.7%	0.920	0.910	0.969
3×10^{-5}	0.825	20.68	0.097	9.9%	0.921	0.910	0.970
10^{-4}	0.247	8.84	0.226	12.8%	0.921	0.911	0.971
3×10^{-4}	0.0825	5.44	0.368	16.6%	0.921	0.911	0.972
10^{-3}	0.0247	3.99	0.501	20.1%	0.921	0.911	0.972
3×10^{-3}	0.00825	2.95	0.678	22.6%	0.920	0.910	0.971
10^{-2}	0.00247	2.12	0.943	26.1%	0.919	0.908	0.969
3×10^{-2}	0.000825	1.59	1.255	30.9%	0.916	0.907	0.968
10^{-1}	0.000247	1.09	1.840	38.9%	0.904	0.894	0.964

bandwidth γ in the grid: the RBF kernel prefers to be nearly linear on these features, so the extension adds little over the linear SVM. The kernel-approximation error falls with a fitted log–log slope of -0.49 (theory: -0.5 , i.e. $1/\sqrt{D}$), and validation AUC keeps improving up to the largest D .

C. Benchmark against scikit-learn

Tables 4–6 are the main results. *Task A:* our tree’s test RMSE is 0.224 (CV 0.232 ± 0.003), scikit-learn’s is 0.224 (CV 0.232 ± 0.003), and the constant baseline’s is 0.628. The back-transformed error is 44,463 AZN MAE (MAPE 15.6%). The bonus ridge regression (0.231) shows how much of the signal is close to linear in log space: the single tree beats it. *Task B:* the best of our models is our decision tree (F1 0.925), while the best model overall is sklearn DecisionTreeClassifier (F1 0.925). The better of our two SVMs trails our tree, and the RFF kernel falls short of the linear SVM (test F1 -0.006). By ROC-AUC the order flips (0.928 for the tree against 0.973 for the SVM): the selected tree grows nearly pure leaves, so its probabilities are almost binary and rank listings poorly, while SVM margins grade them smoothly. Averaged Pegasos reaches F1 0.915 against 0.915 for LIBLINEAR on the same objective.

Structure. Table 8 compares the fitted trees node by node (src/tree_compare.py). At each node we check whether both libraries split the same training rows. If they do not, we classify the difference: an exact tie (two splits with equal gain; scikit-learn breaks ties by visiting features in random order), a *float32 divergence* (scikit-learn casts X to float32, which here merges 33,547 distinct float64 values; we re-run our own split search on the float32-rounded rows of that node and it returns exactly scikit-learn’s split), or an unexplained difference. We find 0 (Task A) and 0 (Task B) unexplained differences, 85 ties and 5 float32 divergences in Task A, and test-prediction agreement of 98.1% and 98.9%. Both trees choose the same root split (dist_center_km ≤ 9.094).

Speed. Our trees fit in 25.53 s vs. 1.416 s for scikit-learn’s Cython code. Our split search is asymptotically the same $O(dm \log m)$ per node, but it pays Python overhead per node and re-sorts at every node instead of reusing a presorted order. Pegasos costs $O(\text{epochs} \cdot nd)$ and fits in 47.32 s. The exact-kernel SVC is $O(n^2) - O(n^3)$ (49.16 s), the main practical argument for RFF.

Scaling. To check these orders empirically we refit every model at its selected configuration on nested training sub-

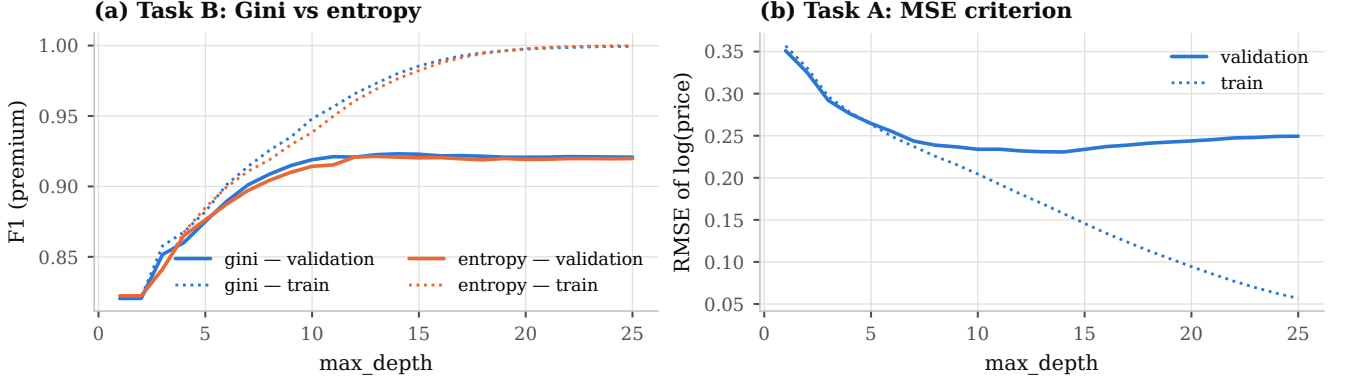
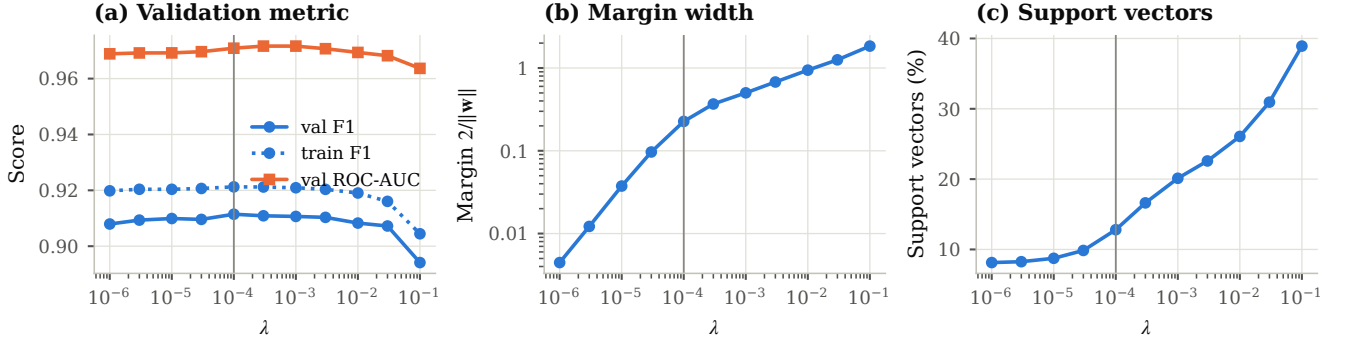
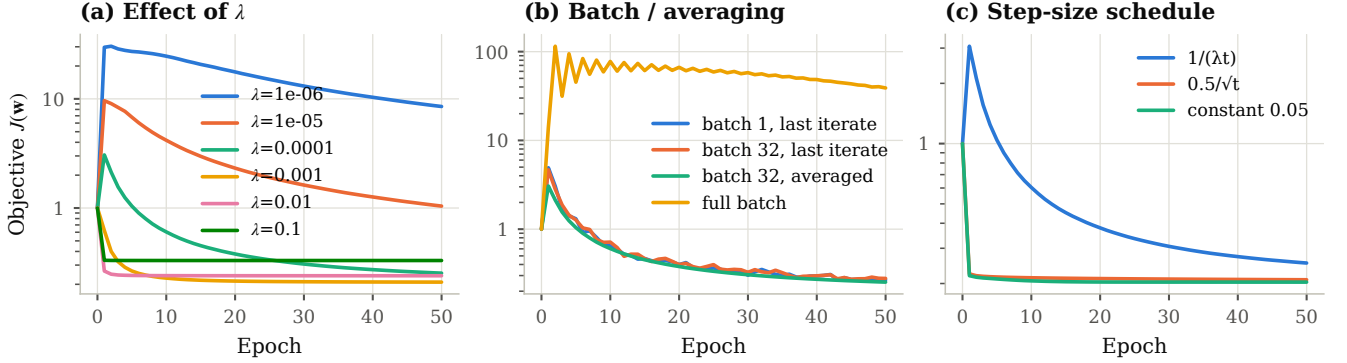


Figure 2. Depth sweep with `min_samples_leaf=1` (dotted: train, solid: validation). Validation error is U-shaped, the bias–variance trade-off. The grey line marks the depth chosen on validation.



Top: effect of λ on (a) validation F1/AUC, (b) the margin width $2/\|w\|$, (c) the share of support vectors.



Middle: training objective per epoch for (a) different λ , (b) batch size and iterate averaging at the selected λ , (c) step-size schedules.

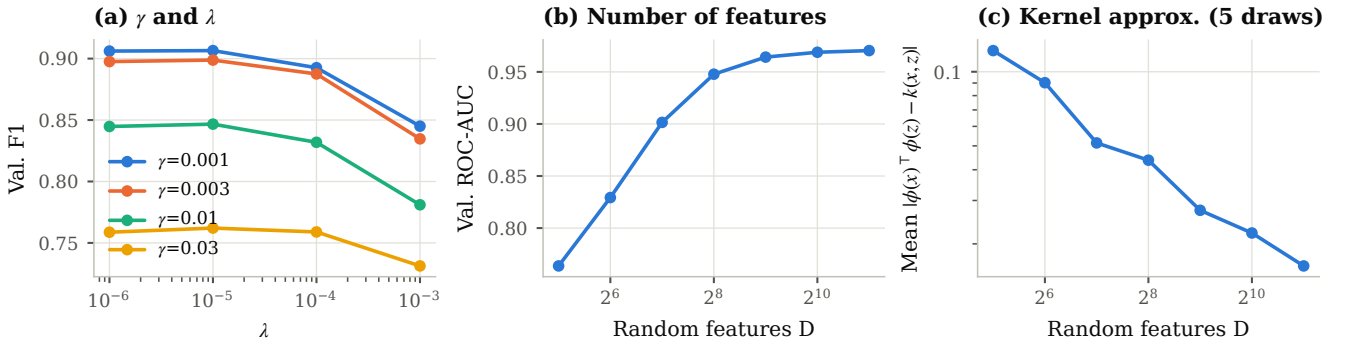


Figure 3. Our Pegasos SVM. **Bottom:** RFF extension: (a) bandwidth $\gamma \times \lambda$, (b) validation AUC vs. number of random features D , (c) mean absolute error of the kernel approximation.

Table 4. Task A, regression of log(price). CV: 5-fold mean $\pm$ std on train+validation. Test: single held-out split with a 95% bootstrap CI. MAE (AZN) and MAPE after back-transformation. Best test RMSE in bold.

Model	CV RMSE (log)	CV R^2 (log)	Test RMSE (log) [95% CI]	Test R^2	Test MAE (AZN)	MAPE	Fit (s)
Constant baseline (train median)	—	—	0.628	-0.001	123,956	54.2%	0.00
Ours: decision tree	0.232 $\pm$ 0.003	0.869 $\pm$ 0.003	0.224 [0.216, 0.232]	0.873	44,463	15.6%	25.53
sklearn DecisionTreeRegressor	0.232 $\pm$ 0.003	0.869 $\pm$ 0.003	0.224 [0.217, 0.233]	0.872	45,369	15.6%	1.42
Ours: ridge (bonus)	0.239 $\pm$ 0.002	0.861 $\pm$ 0.003	0.231 [0.224, 0.237]	0.865	47,491	16.5%	0.51
sklearn Ridge	0.239 $\pm$ 0.002	0.861 $\pm$ 0.003	0.231 [0.224, 0.237]	0.865	47,491	16.5%	0.29

Table 5. Task B, price tier (premium = positive class). Same protocol as Table 4. Best test F1 in bold. † trained on a 20,000-row subsample ($O(n^2)$ kernel solver).

Model	CV F1	CV ROC-AUC	Acc	Prec	Rec	Test F1 [95% CI]	AUC [95% CI]	PR-AUC	Fit (s)
Ours: decision tree	0.923 $\pm$ 0.003	0.917 $\pm$ 0.003	0.925	0.930	0.919	0.925 [0.919, 0.930]	0.928 [0.922, 0.935]	0.914	52.00
sklearn DecisionTreeClassifier	0.922 $\pm$ 0.004	0.916 $\pm$ 0.004	0.926	0.930	0.920	0.925 [0.920, 0.931]	0.930 [0.924, 0.936]	0.916	2.38
Ours: linear SVM (Pegasos)	0.917 $\pm$ 0.003	0.974 $\pm$ 0.001	0.915	0.907	0.924	0.915 [0.909, 0.922]	0.973 [0.970, 0.976]	0.974	47.32
sklearn SGDClassifier (hinge)	0.905 $\pm$ 0.003	0.965 $\pm$ 0.001	0.897	0.890	0.906	0.898 [0.892, 0.905]	0.964 [0.960, 0.967]	0.965	2.55
sklearn LinearSVC	0.917 $\pm$ 0.003	0.974 $\pm$ 0.001	0.914	0.905	0.924	0.915 [0.909, 0.921]	0.973 [0.970, 0.976]	0.974	82.82
Ours: RBF SVM (RFF + Pegasos)	0.913 $\pm$ 0.003	0.972 $\pm$ 0.001	0.909	0.903	0.916	0.910 [0.904, 0.916]	0.970 [0.967, 0.973]	0.970	15.52
sklearn SVC (RBF) †	0.915 $\pm$ 0.004	0.973 $\pm$ 0.001	0.910	0.904	0.918	0.911 [0.905, 0.917]	0.971 [0.968, 0.974]	0.971	49.16

Table 6. Headline results on both tasks (Task A: RMSE of log price, lower is better; Task B: F1, higher is better).

Model	A: RMSE (CV)	A: test	B: F1 (CV)	B: test
Ours: decision tree	0.232 $\pm$ 0.003	0.224	0.923 $\pm$ 0.003	0.925
Ours: linear SVM (Pegasos)	n/a	n/a	0.917 $\pm$ 0.003	0.915
Ours: RBF SVM (RFF + Pegasos)	n/a	n/a	0.913 $\pm$ 0.003	0.910
Ours: ridge (bonus)	0.239 $\pm$ 0.002	0.231	n/a	n/a
sklearn DecisionTree*	0.232 $\pm$ 0.003	0.224	0.922 $\pm$ 0.004	0.925
sklearn LinearSVC	n/a	n/a	0.917 $\pm$ 0.003	0.915
sklearn SVC (RBF)	n/a	n/a	0.915 $\pm$ 0.004	0.911

Table 7. Fit time vs. training size: exponent b of $t \approx an^b$ (three largest subsamples), and fit/predict time at the largest size (predict on the validation split). Theory: CART $O(n \log n)$, Pegasos/SGD/LIBLINEAR $O(n)$ per epoch, kernel SVC $O(n^2) - O(n^3)$.

Model	Exponent b	Largest n	Fit (s)	Predict (s)
Ours: tree (A)	1.36	40,405	20.82	0.008
sklearn tree (A)	1.39	40,405	1.61	0.007
Ours: tree (B)	1.31	40,405	39.94	0.011
sklearn tree (B)	1.24	40,405	1.74	0.007
Ours: Pegasos	1.05	40,405	4.26	0.001
sklearn SGDClassifier	1.46	40,405	2.06	0.003
sklearn LinearSVC	0.21	40,405	64.13	0.005
Ours: RFF + Pegasos	0.90	40,405	12.12	0.284
sklearn SVC (RBF)	2.72	14,142	16.57	7.632

samples of 2,020–40,405 rows (src/scaling.py) and fit $t \approx an^b$ through the three largest sizes (Table 7, Fig. 15). Fit time grows with exponent $b = 1.36$ for our Task A tree (scikit-learn: 1.39), 1.05 for Pegasos, 0.90 for RFF + Pegasos and 2.72 for the exact-kernel SVC. The trees and the Pegasos models stay close to the $O(n \log n)/O(n)$ of the analysis, so the gap to scikit-learn is a constant factor (our tree is 12.9 $\times$ slower at full size on Task A, 23.0 $\times$ on Task B), not a worse algorithm. The kernel SVC is the only super-linear method; at this dataset’s size that is what forces it onto a subsample.

D. Are the differences real?

Several gaps in Tables 4 and 5 are in the third decimal, and a single test split could produce them by chance. To check,

Table 8. Our tree vs. scikit-learn, node by node (final models; depth and leaves as ours / sklearn). Same: identical split. Ties: equal-gain alternatives. f32: explained by scikit-learn’s float32 cast. Other: unexplained (0 for a correct implementation).

Task	Depth	Leaves	Same	Ties	f32	Other	Agree
A (MSE)	14 / 14	743 / 745	624	85	5	0	98.1%
B (gini)	14 / 14	1,609 / 1,610	889	372	9	0	98.9%

we resample the 8,658 test rows with replacement 1,000 times and score *both* models of a pair on the same resample. Rows that are hard for one model are usually hard for the other, so this shared noise cancels in the difference, and its interval is much narrower than the two per-model intervals in Tables 4–5 would suggest. A difference counts as real when its 95% percentile interval excludes zero; p is the two-sided bootstrap p -value $2 \min\{P(d^* \leq 0), P(d^* \geq 0)\}$ (evaluate.paired_bootstrap).

Table 9 lists all 20 comparisons, of which 10 are significant, including 2 of the 12 comparisons of our code with scikit-learn. *Task A*: our tree’s test RMSE minus scikit-learn’s is -0.0007 [$-0.0021, +0.0002$] (not significant), and our forest’s minus scikit-learn’s is -0.0002 [$-0.0015, +0.0012$] (not significant). The forest’s gain over our single tree is -0.0498 [$-0.0546, -0.0452$] (significant). *Task B*: averaged Pegasos minus LIBLINEAR is $+0.0008$ F1 [$-0.0005, +0.0019$] (not significant), and the RFF SVM minus the exact kernel SVC is -0.0008 ROC-AUC [$-0.0018, +0.0001$] (not significant). The kernel’s gain over the linear SVM is -0.0058 F1 (significant) and -0.0029 ROC-AUC (significant). Our tree minus the RFF SVM is $+0.0149$ F1 (significant) and -0.0420 ROC-AUC (significant), so the “tree wins F1, SVM wins AUC” split can be read off with its uncertainty. A significant difference is not automatically a meaningful one: with 8,658 rows, gaps of a few thousandths of RMSE can be detected but are far below what a buyer would notice.

Table 9. Paired bootstrap comparisons on the test split. Difference = first – second (lower is better for RMSE, higher for F1 and ROC-AUC; R^2 is omitted because on a fixed test sample it is a monotone function of RMSE), with its 95% percentile interval over 1,000 resamples of the test rows. “No real difference”: the interval contains 0.

Task	Comparison (first vs second)	Metric	First	Second	Difference [95% CI]	p	Verdict
A	Our tree vs sklearn tree	RMSE	0.2237	0.2244	−0.0007 [−0.0021, +0.0002]	0.190	no real difference
A	Our ridge vs sklearn ridge	RMSE	0.2308	0.2308	0.0000 [0.0000, 0.0000]	0.260	no real difference
A	Our tree vs our ridge	RMSE	0.2237	0.2308	−0.0071 [−0.0139, −0.0012]	0.028	Our tree better
B	Our tree vs sklearn tree	F1	0.9246	0.9251	−0.0005 [−0.0027, +0.0016]	0.606	no real difference
B	Our tree vs sklearn tree	ROC-AUC	0.9284	0.9296	−0.0012 [−0.0044, +0.0019]	0.418	no real difference
B	Pegasos vs LinearSVC	F1	0.9155	0.9147	+0.0008 [−0.0005, +0.0019]	0.240	no real difference
B	Pegasos vs LinearSVC	ROC-AUC	0.9733	0.9731	+0.0002 [0.0000, +0.0004]	0.096	no real difference
B	RFF SVM vs SVC (RBF)	F1	0.9097	0.9110	−0.0014 [−0.0047, +0.0020]	0.406	no real difference
B	RFF SVM vs SVC (RBF)	ROC-AUC	0.9704	0.9712	−0.0008 [−0.0018, +0.0001]	0.084	no real difference
B	RFF SVM vs Pegasos	F1	0.9097	0.9155	−0.0058 [−0.0096, −0.0020]	<0.001	Pegasos better
B	RFF SVM vs Pegasos	ROC-AUC	0.9704	0.9733	−0.0029 [−0.0041, −0.0016]	<0.001	Pegasos better
B	Our tree vs RFF SVM	F1	0.9246	0.9097	+0.0149 [+0.0087, +0.0214]	<0.001	Our tree better
B	Our tree vs RFF SVM	ROC-AUC	0.9284	0.9704	−0.0420 [−0.0483, −0.0362]	<0.001	RFF SVM better
A	Our forest vs sklearn forest	RMSE	0.1739	0.1740	−0.0002 [−0.0015, +0.0012]	0.834	no real difference
A	Our boosting vs sklearn boosting	RMSE	0.2002	0.2005	−0.0003 [−0.0012, +0.0007]	0.574	no real difference
A	Our forest vs our single tree	RMSE	0.1739	0.2237	−0.0498 [−0.0546, −0.0452]	<0.001	Our forest better
B	Our forest vs sklearn forest	F1	0.9452	0.9422	+0.0029 [0.0000, +0.0058]	0.050	Our forest better
B	Our forest vs sklearn forest	ROC-AUC	0.9888	0.9880	+0.0008 [+0.0001, +0.0015]	0.014	Our forest better
B	Our forest vs our single tree	F1	0.9452	0.9246	+0.0206 [+0.0160, +0.0255]	<0.001	Our forest better
B	Our forest vs our single tree	ROC-AUC	0.9888	0.9284	+0.0604 [+0.0544, +0.0664]	<0.001	Our forest better

E. Error analysis

What the models use. Our tree’s impurity-based importances (Fig. 4) are led by `dist_center_km` (0.59), `land_area_sot` (0.10), `lng` (0.08), `area_m2` (0.04) for Task A and by `area_m2` (0.54), `dist_center_km` (0.15), `lng` (0.08), `land_area_sot` (0.04) for Task B. Size and location dominate, which matches the EDA. In the linear SVM (Fig. 6), whose weights are comparable because the features are standardised, the largest positive weights are `log_area`, `location=ag seher q.`, `location=nizami m.`, `total_floors` and the most negative are `dist_center_km`, `area_m2`, `location=bineqedi q.`, `is_top_floor`. Correlated features share their effect (area and log area enter together), so a single weight, such as a negative one on raw area next to a large positive one on log area, must not be read in isolation.

Where the regression fails. (i) *Luxury and bargain outliers.* The tree under-predicts the top price decile (mean log residual −0.093) and over-predicts the bottom one (0.118, Fig. 5, bottom left). Leaves average many listings, which pulls the extremes towards the mean; this is the variance-reduction side of the bias–variance trade-off. Table 10 lists the largest errors. (ii) *Sparse districts* (Fig. 5, top middle). The test MAE is 0.365 for listings whose district has <20 training examples, against 0.131 for 500+: rare districts fall into the “other” column and fall back on coordinates. (iii) *Categories.* The error is highest for `heyet evi/bag evi` (0.281) and lowest for `kohne tikili` (0.120). 4.4% of test listings are off by more than 50% in price.

Where the classifier fails. Tier errors concentrate at the boundary. 42% of the tree’s misclassifications (SVM: 53%) are listings within $\pm 10\%$ of the threshold price, a band that holds only 11% of the test rows. The error rate falls from 35.0% closest to the threshold to 1.7% far from it (Fig. 5, bottom right). Many of these “errors” are therefore near-ties that no feature could resolve: a listing just above the median

Table 10. Largest Task A test errors of our tree.

True (AZN)	Pred. (AZN)	m ²	Rooms	Category	Location
180,000	18,797	16	6	heyet evi/bag evi	nzs q.
85,000	623,314	500	4	heyet evi/bag evi	buzovna q.
1,300,000	201,448	300	11	heyet evi/bag evi	novxani q.
499,000	2,809,578	650	6	heyet evi/bag evi	merdekan q.
199,000	36,927	27	2	heyet evi/bag evi	elmier akademiyasi m.
1,020,000	215,485	258	4	heyet evi/bag evi	buzovna q.

is not meaningfully more premium than one just below it.

F. Feature importance across models

MDI importances exist only for trees and are measured on training data. To compare every model on the same footing we use *permutation importance* on the test split: shuffle one feature across rows, re-score the fitted model, and record how much worse it gets (Task A: increase in RMSE of log price; Task B: drop in ROC-AUC; mean of 3 shuffles). The 292 design columns are grouped into 29 raw features, so all one-hot columns of `district` or `location` are shuffled together and a categorical feature is not split across dozens of small columns; area and log area form one group because they carry the same information. No model is refitted.

For Task A (Fig. 7, left), our random forest relies most on area (RMSE +0.527), `dist_center_km` (RMSE +0.142), `lng` (RMSE +0.078), `land_area_sot` (RMSE +0.048). All 4 models rank area first. For Task B (right) the RFF SVM relies most on area (AUC −0.152), `tags` (AUC −0.066), `rooms` (AUC −0.027), `location` (AUC −0.024). All 3 models rank area first. Two caveats apply. Correlated features (area and area per room; latitude, longitude and distance to the centre) can stand in for each other, so shuffling one of them understates how much the model needs the shared information. And the importance of a feature is a property of the model, not of the market: a feature a model ignores may still be predictive.

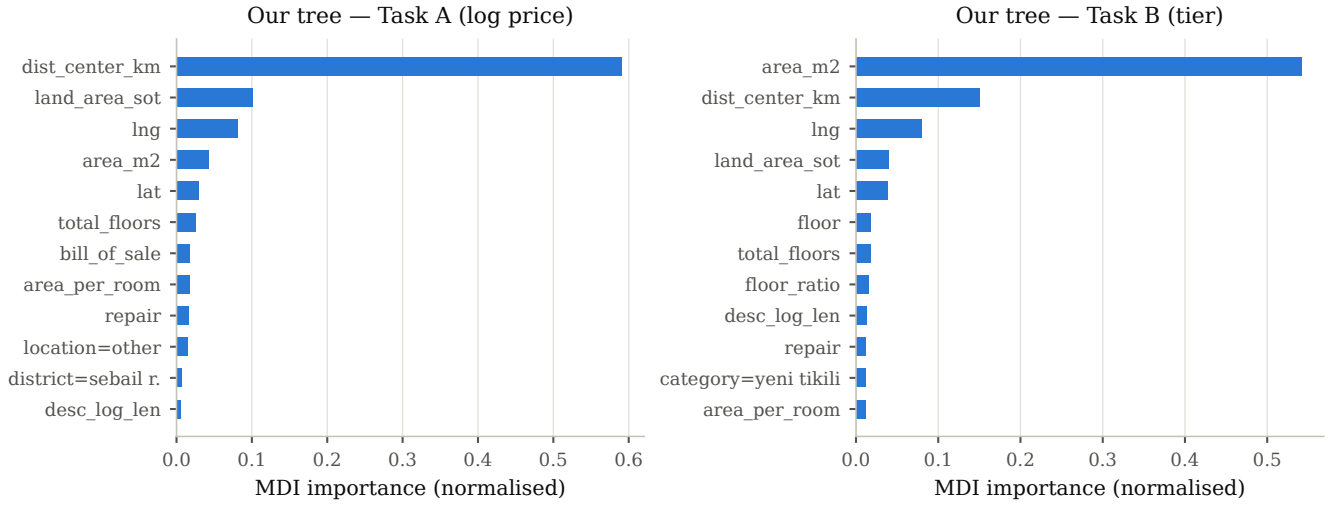


Figure 4. Top impurity-decrease (MDI) importances of our final trees.

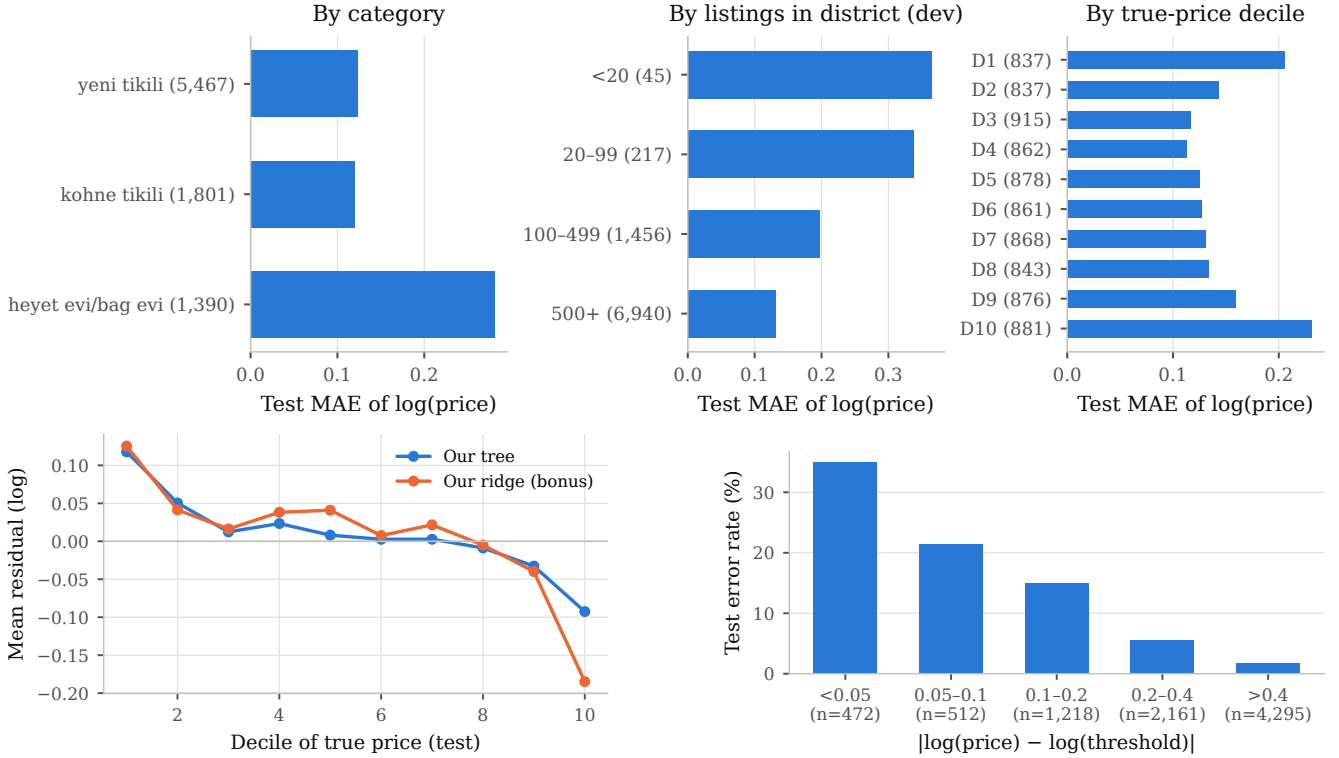


Figure 5. Error analysis on the test split. Top: Task A MAE of log(price) by category, by number of training listings in the district, and by true-price decile (row counts in brackets). Bottom left: mean residual per price decile; the tree pulls the extremes towards the mean. Bottom right: Task B error rate vs. distance from the threshold. A map of the residuals is in the appendix (Fig. 10).

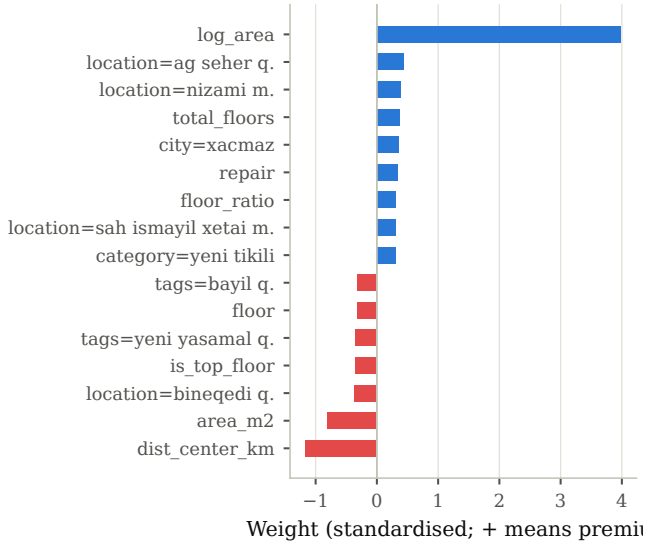


Figure 6. Largest weights of the final linear SVM (standardised features; positive values push towards premium).

Table 11. Test error by district. RMSE of log price for Task A models; tier error rate (err.) for Task B models.

District	n	Tree	Ridge	RF	GB	Tree err.	SVM err.
xetai r.	1,542	0.170	0.179	0.126	0.150	8.7%	10.4%
yasamal r.	1,455	0.196	0.200	0.144	0.178	7.1%	9.8%
nesimi r.	1,258	0.175	0.177	0.136	0.160	6.4%	6.4%
nerimanov r.	1,061	0.170	0.177	0.129	0.164	6.4%	8.1%
bineqedi r.	691	0.215	0.233	0.169	0.194	7.7%	9.7%
nizami r.	587	0.148	0.162	0.109	0.143	6.8%	9.2%
xezer r.	579	0.332	0.332	0.270	0.285	10.0%	10.2%
abseron r.	454	0.319	0.298	0.250	0.268	4.2%	5.5%
sebail r.	398	0.334	0.364	0.276	0.315	11.6%	16.6%
sabuncu r.	305	0.369	0.412	0.282	0.322	11.8%	11.5%
suraxani r.	289	0.260	0.232	0.198	0.220	3.5%	3.8%
other	39	0.288	0.309	0.266	0.261	5.1%	0.0%
All test	8,658	0.224	0.231	0.174	0.200	7.5%	9.1%

G. Where the models are weakest

Fig. 8 and Tables 11–12 split the test error by named district (the *r.* tag on bina.az; districts with fewer than 30 test listings are pooled as “other”) and by true price band. For our random forest (overall RMSE 0.174), the weakest district is sabuncu *r.* (RMSE 0.282 on 305 listings) and the strongest is nizami *r.* (0.109). By price, the error is largest for 600k+ AZN (0.301) and smallest for 200k–350k AZN (0.141). The mean residual of our random forest is +0.123 in the cheapest band and −0.103 in the most expensive one. Every model pulls the extremes towards the middle: cheap listings are over-priced and luxury ones under-priced. Which districts are hard is largely shared across models (mean pairwise Spearman correlation of per-district RMSE 0.98), so the weak spots lie mainly in the data, not in one model. For Task B the RFF SVM’s error rate is highest in sebail *r.* (16.6%) and in the 200k–350k AZN band (15.5%, the band that contains the tier threshold).

H. Bonus: ridge, SVR, ensembles of our trees, clustering

Ridge (normal equations, $\alpha = 100$ chosen on validation) matches scikit-learn’s Ridge to machine precision. *SVR* (`src/svr.py`) turns our SVM into a Task A model: the same Pegasos step on the ε -insensitive primal

Table 12. Test error by true price band (columns as in Table 11).

Price (AZN)	n	Tree	Ridge	RF	GB	Tree err.	SVM err.
<75k	407	0.349	0.365	0.283	0.317	0.5%	0.2%
75k–125k	1,160	0.235	0.232	0.183	0.205	1.9%	1.9%
125k–200k	2,571	0.176	0.189	0.142	0.158	8.4%	12.2%
200k–350k	2,763	0.191	0.180	0.141	0.165	13.9%	15.5%
350k–600k	1,314	0.237	0.243	0.182	0.221	1.5%	1.6%
600k+	443	0.384	0.440	0.301	0.349	0.9%	0.7%
All test	8,658	0.224	0.231	0.174	0.200	7.5%	9.1%

Table 13. Bonus models on Task A (test split).

Task A model (test)	RMSE (log)	R^2	Fit (s)
Ours: single decision tree	0.224	0.873	–
Ours: ridge (normal equations)	0.231	0.865	–
Ours: random forest	0.174	0.923	1,118.7
sklearn RandomForest	0.174	0.923	34.0
Ours: gradient boosting	0.200	0.898	1,498.2
sklearn GradientBoosting	0.201	0.898	132.4

Table 14. Bonus: support-vector regression on Task A (test split), ours vs. scikit-learn with matched $C = 1/(n\lambda)$, ε and γ . [‡]Trained on a 20,000-row subsample (quadratic kernel solver).

Task A model (test)	RMSE (log)	R^2	MAE (AZN)	Fit (s)
Ours: linear SVR	0.231	0.865	47,612	2.40
sklearn LinearSVR	0.231	0.865	47,651	402.23
Ours: RBF SVR (RFF)	0.245	0.848	52,156	12.09
sklearn SVR (RBF) [‡]	0.225	0.872	46,043	65.08

$\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \max(0, |y_i - \mathbf{w}^\top \mathbf{x}_i| - \varepsilon)$, whose sub-gradient is $-\text{sign}(r_i) \mathbf{x}_i$ outside the tube and 0 inside. Because a regressor’s predictions depend on the *scale* of $\mathbf{w}$, not only its sign, the first $1/\lambda$ Pegasos steps are too large to settle within our epoch budget, so we offset the schedule to $\eta_t = 1/(\lambda(t + t_0))$ with $\eta_1 = 1/\|\mathbf{x}\|^2$. Chosen on validation: $\lambda = 10^{-4}$, $\varepsilon = 0.10$ (in log units); RFF $\gamma = 0.000$, $\lambda = 10^{-5}$. On the test split the linear SVR reaches RMSE 0.231 (LIBLINEAR LinearSVR: 0.231; objective gap −0.05%) and the RFF SVR 0.245 (SVR with the exact RBF kernel on a 20,000-row subsample: 0.225; Table 14). The absolute loss makes the SVR robust to the luxury tail, but on log price that tail is already compressed: the linear SVR is on a par with ridge (0.231), and neither SVR reaches the forest. *Ensembles* (Fig. 13, Table 13): a random forest of 60 of our trees (bootstrap + a third of the features per split) lowers the test RMSE: 0.174 against 0.224 for a single tree (scikit-learn’s forest: 0.174). Least-squares gradient boosting with 120 depth-4 trees reaches 0.200 (scikit-learn: 0.201). The boosting curve falls steadily as learners are added. For Task B our forest reaches F1 0.945 (scikit-learn 0.942). *Clustering*: *k*-means++ ($k = 12$) on the projected coordinates recovers neighbourhoods (Fig. 14). Per-cluster test MAE ranges from 0.124 to 0.407 (cluster 3). Adding cluster indicators to the tree lowers the validation RMSE (0.222 $\rightarrow$ 0.221). PCA needs 138 components for 90% of the variance; the first two hold only 6.1%.

VII. DISCUSSION AND LIMITATIONS

Bias and variance. A single deep tree has low bias and high variance. Validation-chosen depth and leaf size cut the vari-

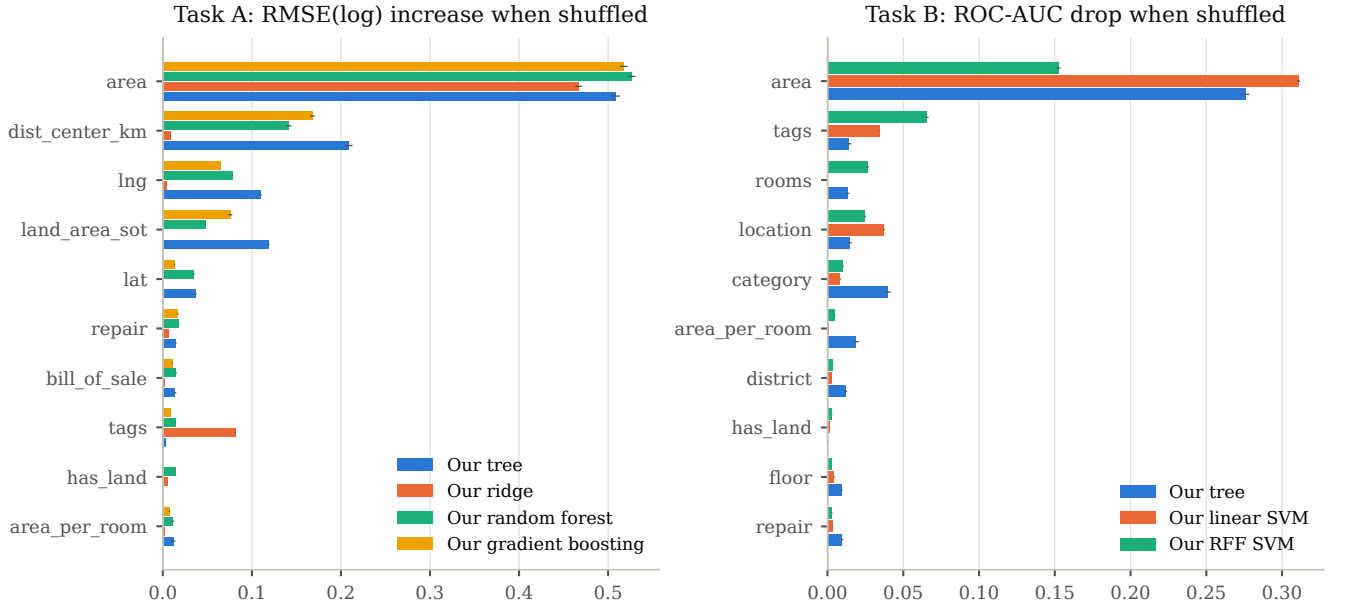


Figure 7. Permutation importance on the test split, per raw feature (one-hot columns grouped). Bars: mean over shuffles; whiskers: std. Features are ranked by the forest (Task A) and the RFF SVM (Task B).

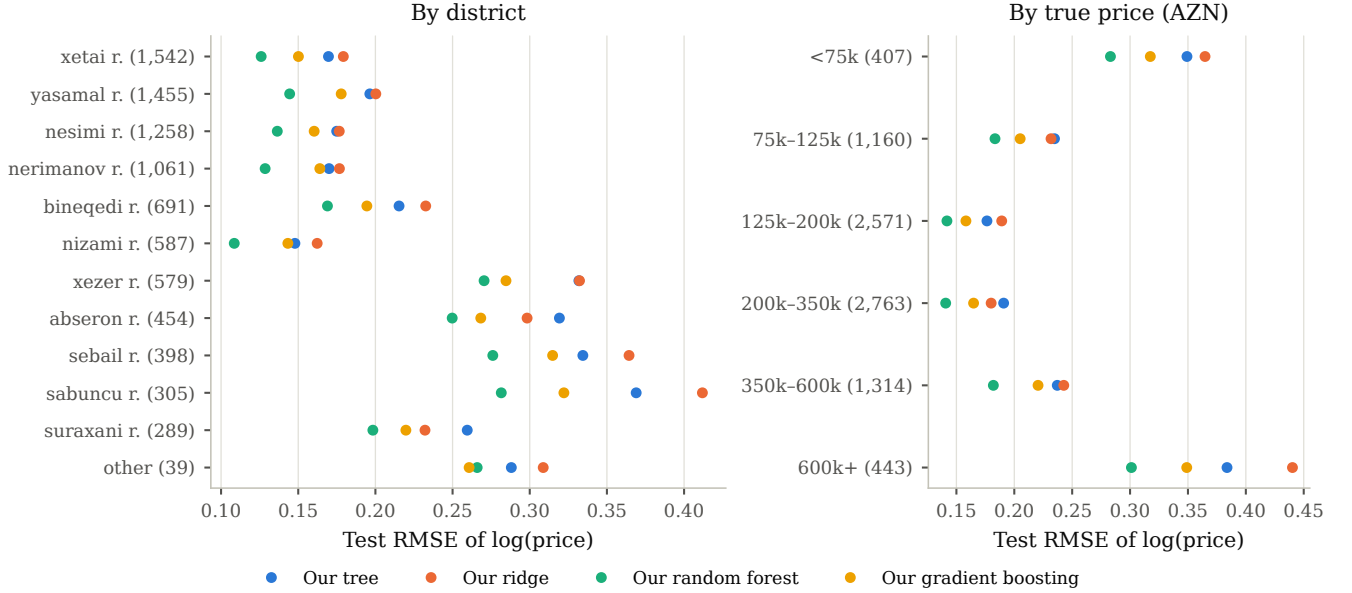


Figure 8. Task A test RMSE of log(price) by district and by true price band, one marker per model (test rows in brackets).

ance, and averaging many decorrelated trees (the forest) cuts it further, which is why our forest beats the single tree. The linear SVM has the opposite profile: it cannot express interactions such as “large *and* central”, and RFF buys that flexibility at the price of tuning γ .

Honest limitations. (1) Asking prices are not transaction prices, and re-posted listings with edited prices can survive de-duplication. (2) Cleaning thresholds and the residential scope are judgement calls. They are fixed and documented but not tuned, and filtering on price per m^2 removes some genuine extremes along with typos. (3) One six-week snapshot and one random split: CV quantifies sampling noise but not temporal drift. A time-based split would be the stricter test. (4) Task B is a median split of Task A’s target, so it in-

herits all of Task A’s noise and has an irreducible error band at the threshold. (5) Our tree is several times slower than scikit-learn’s, and Pegasos with very small λ needs many epochs. The RFF model is a Monte-Carlo approximation whose result depends on the random seed. (6) The description keywords are crude. A proper text model could add signal, but it would also need care to avoid prices written in the text.

VIII. CONCLUSION

Both from-scratch models behave as specified: the tree matches scikit-learn’s split at every node up to ties and float32 effects, and Pegasos reaches LIBLINEAR’s objective up to 0.40%. Both are competitive on real, messy data. Most of the work, and most of the error, lies in the data:

repeated scrapes, duplicates, sparse districts and the luxury tail. Next steps are a time-based split, district-level target encoding learned inside the folds, quantile losses for the tails, and histogram-based split finding to close the speed gap.

CONTRIBUTIONS

See `contribution_report.md` in the repository root for who did what.

TOOLS AND ACKNOWLEDGEMENTS

Data: the Kaggle *Bina.az sale project* dataset [13]; we do not redistribute it. `scikit-learn` [9] is used only for the baselines, the bonus comparisons and the unit tests that check our metrics and models. **AI assistance:** an AI coding assistant (Claude Code, Anthropic) was used to draft and refactor parts of the code, tests and this report. All of it was reviewed, run and checked by the team, and every number in this report is regenerated by `python -m src.run_all`.

REFERENCES

- [1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, “Pegasos: primal estimated sub-gradient solver for SVM,” *Math. Programming*, vol. 127, no. 1, pp. 3–30, 2011.
- [2] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [3] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [5] A. Rahimi and B. Recht, “Random features for large-scale kernel machines,” in *Proc. NeurIPS*, 2007, pp. 1177–1184.
- [6] S. Lacoste-Julien, M. Schmidt, and F. Bach, “A simpler approach to obtaining an $O(1/t)$ convergence rate for the projected stochastic subgradient method,” arXiv:1212.2002, 2012.
- [7] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” in *Proc. COMPSTAT*, 2010, pp. 177–186.
- [8] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, “LIBLINEAR: A library for large linear classification,” *JMLR*, vol. 9, pp. 1871–1874, 2008.
- [9] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [10] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [11] J. H. Friedman, “Greedy function approximation: a gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [12] D. Arthur and S. Vassilvitskii, “k-means++: the advantages of careful seeding,” in *Proc. ACM-SIAM SODA*, 2007, pp. 1027–1035.
- [13] sehriyarmemmedli, “Bina.az sale project,” Kaggle dataset. <https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project>

APPENDIX: ADDITIONAL FIGURES

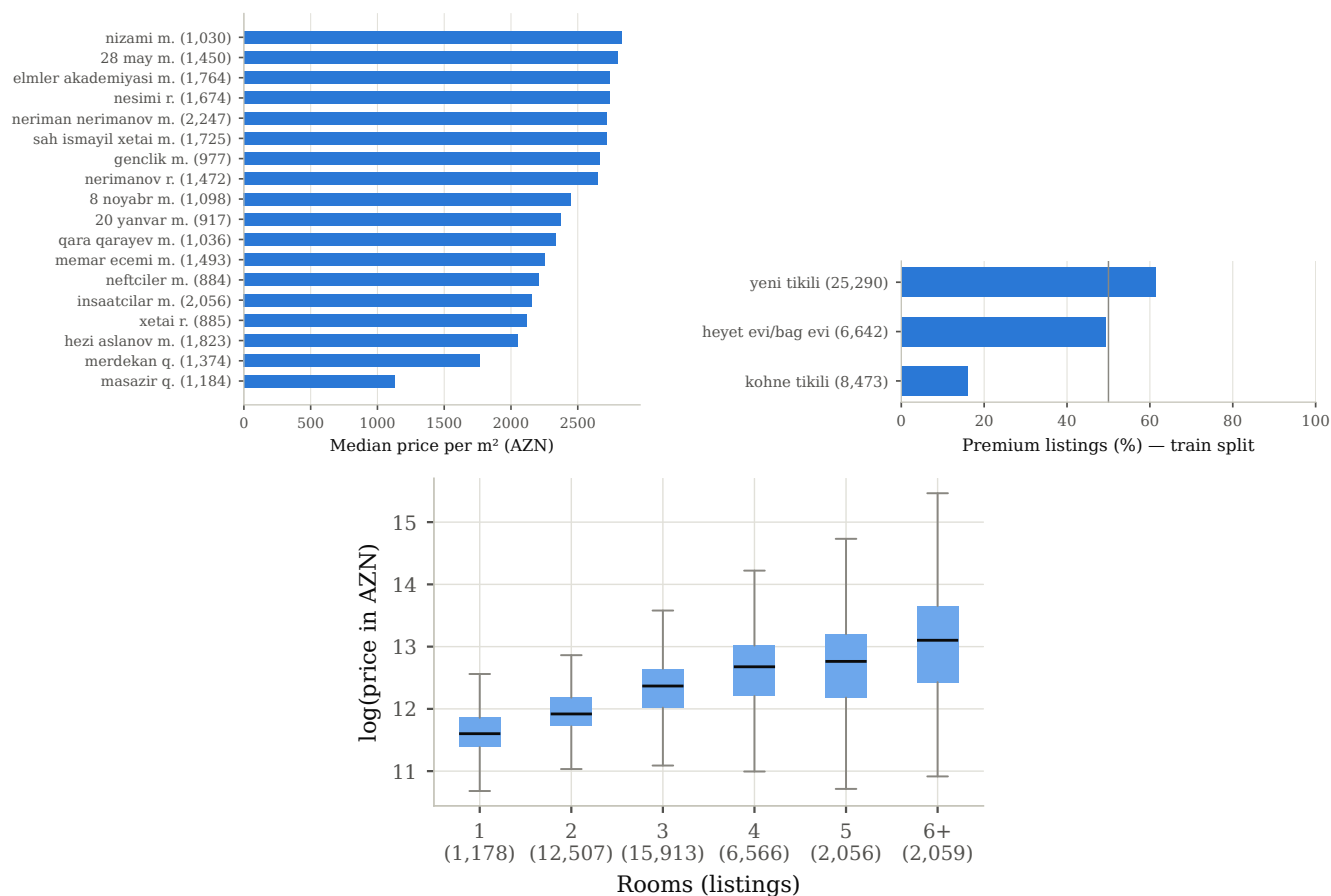


Figure 9. Location effect (median price per m² by district, training listings in brackets), premium share by category, and log price by room count (training split).

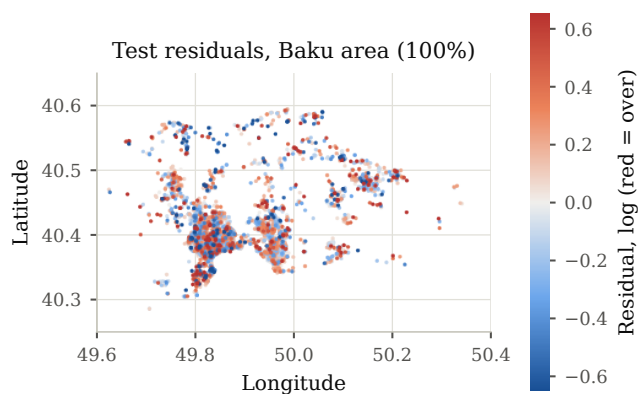


Figure 10. Test residuals of our Task A tree across Baku (red: over-predicted, blue: under-predicted).

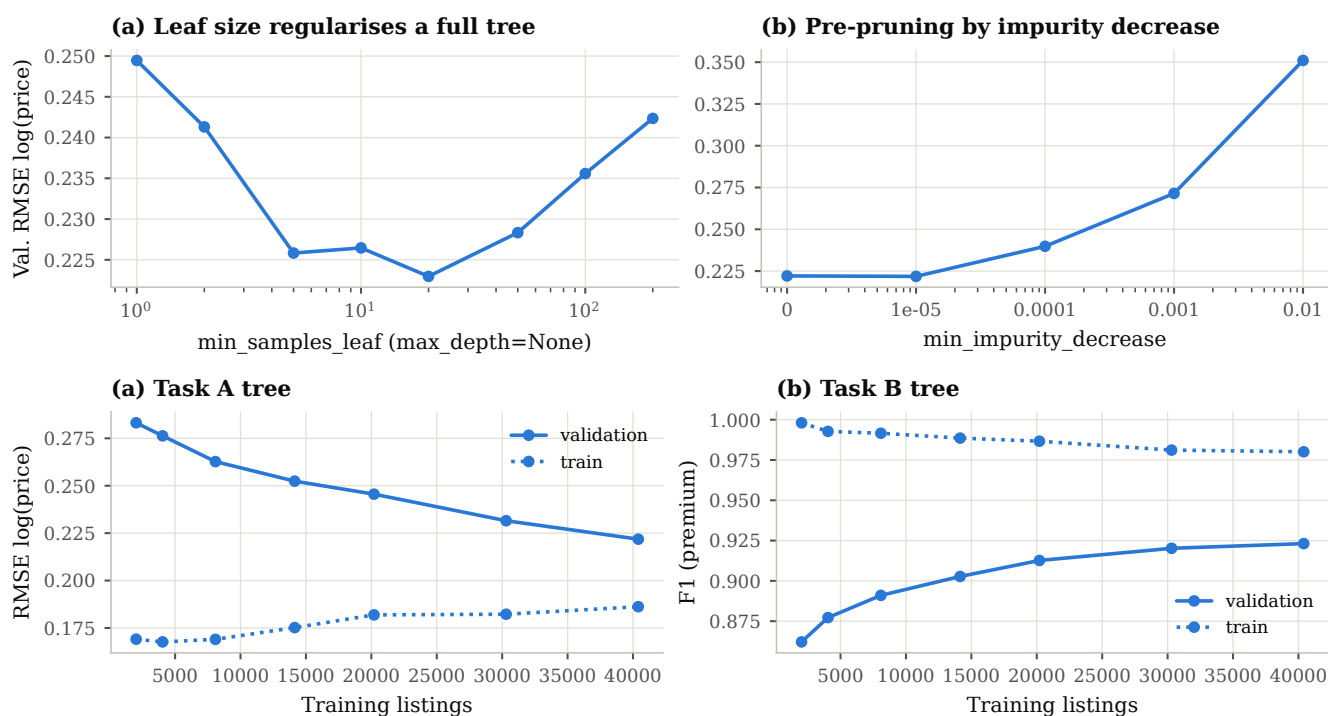


Figure 11. Top: validation RMSE vs. min_samples_leaf for a fully grown tree, and vs. min_impurity_decrease at the selected depth. Bottom: learning curves of the selected trees.

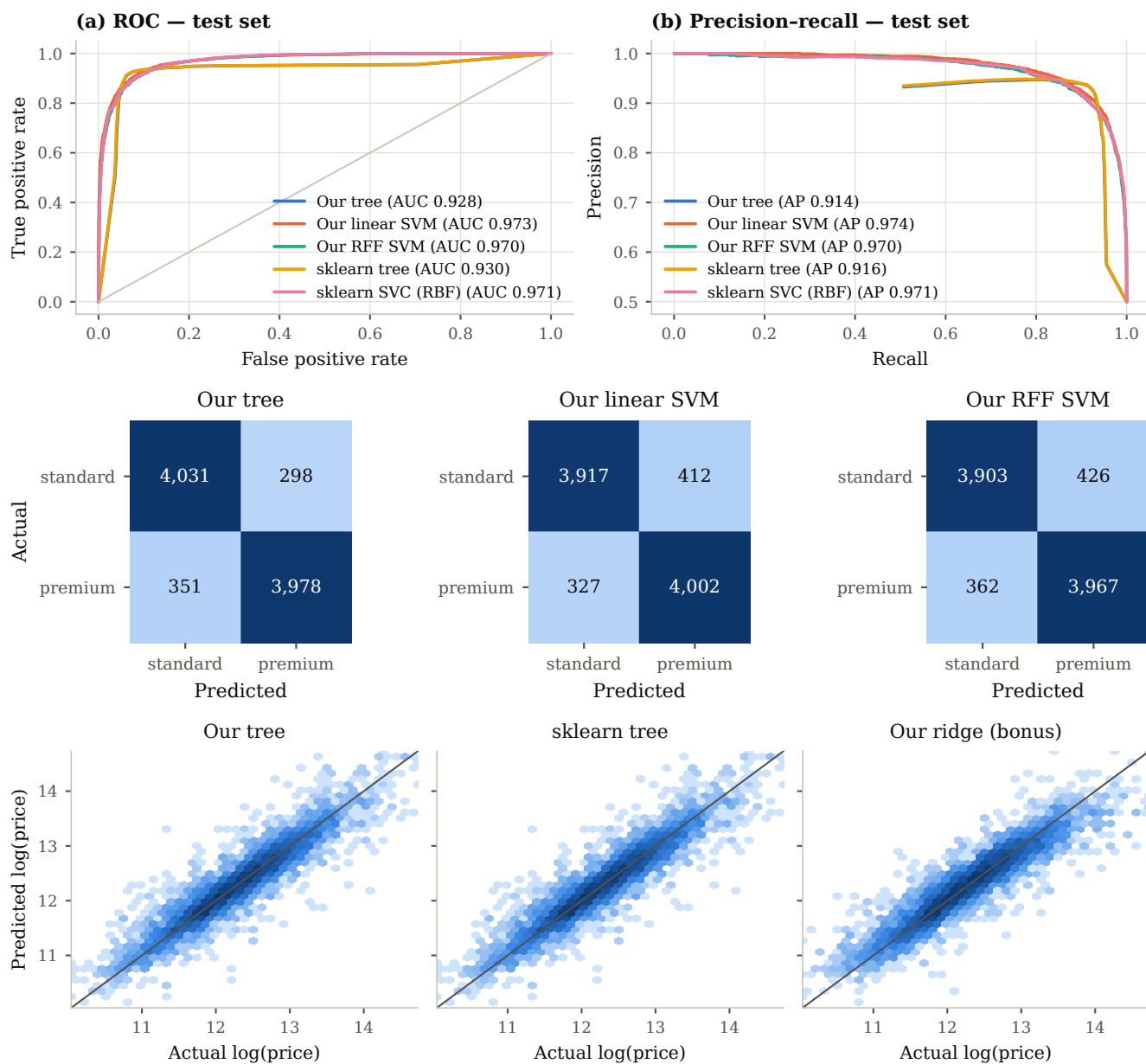


Figure 12. Test-set ROC and precision–recall curves, confusion matrices, and predicted vs. actual log price.

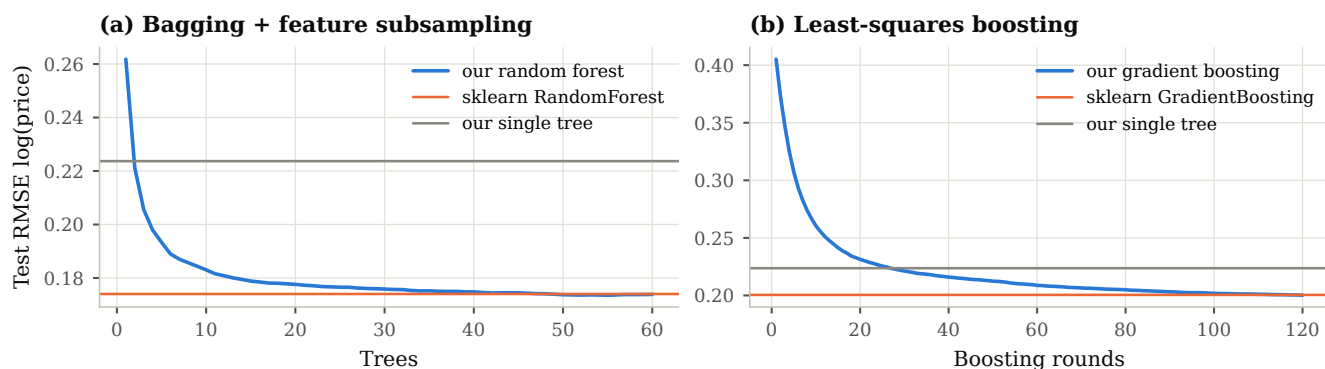


Figure 13. Bonus: test RMSE as learners are added to our random forest (a) and our gradient boosting (b), with scikit-learn's final value and our single tree as references.

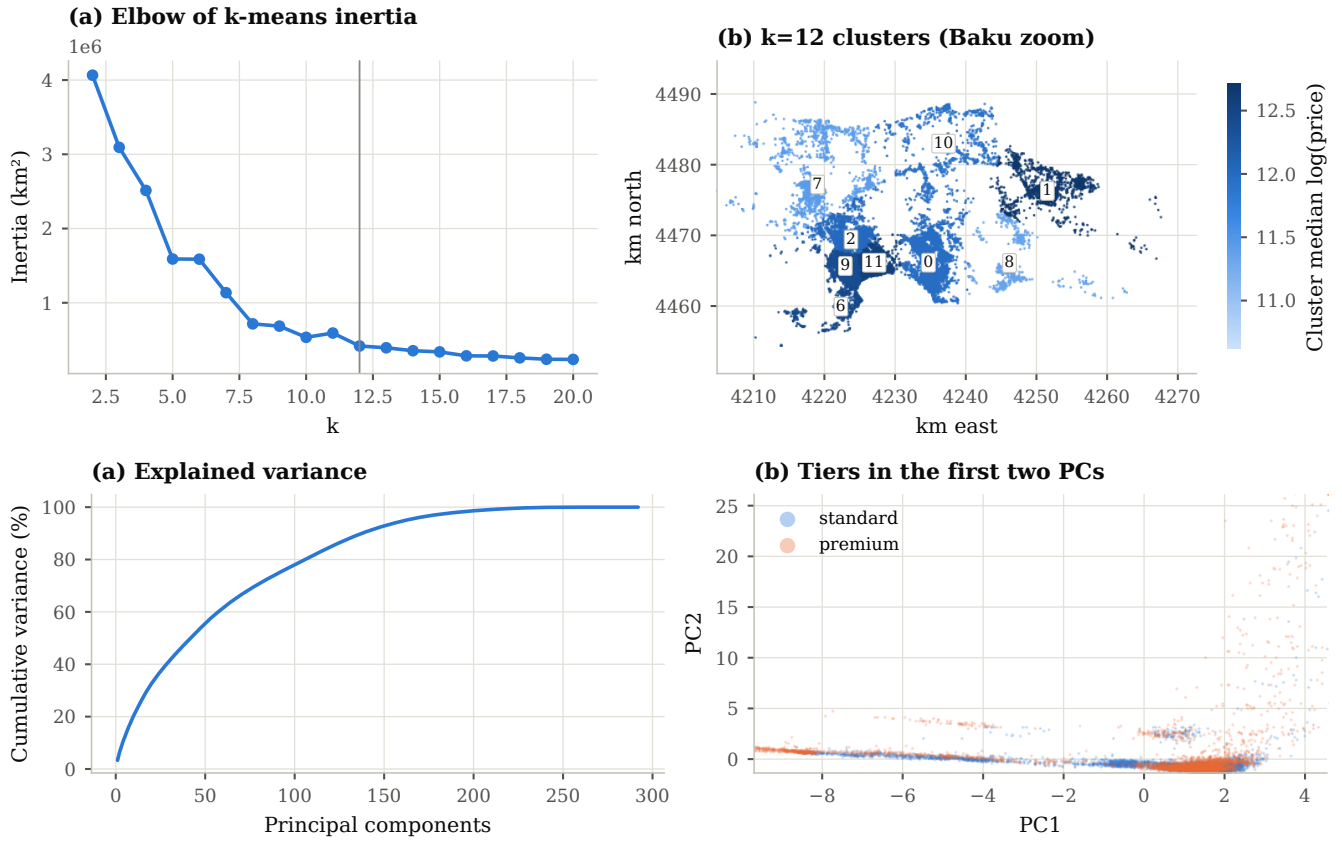


Figure 14. Bonus: k-means elbow and geographic clusters coloured by median log price; PCA explained variance and the two tiers in the first two components.

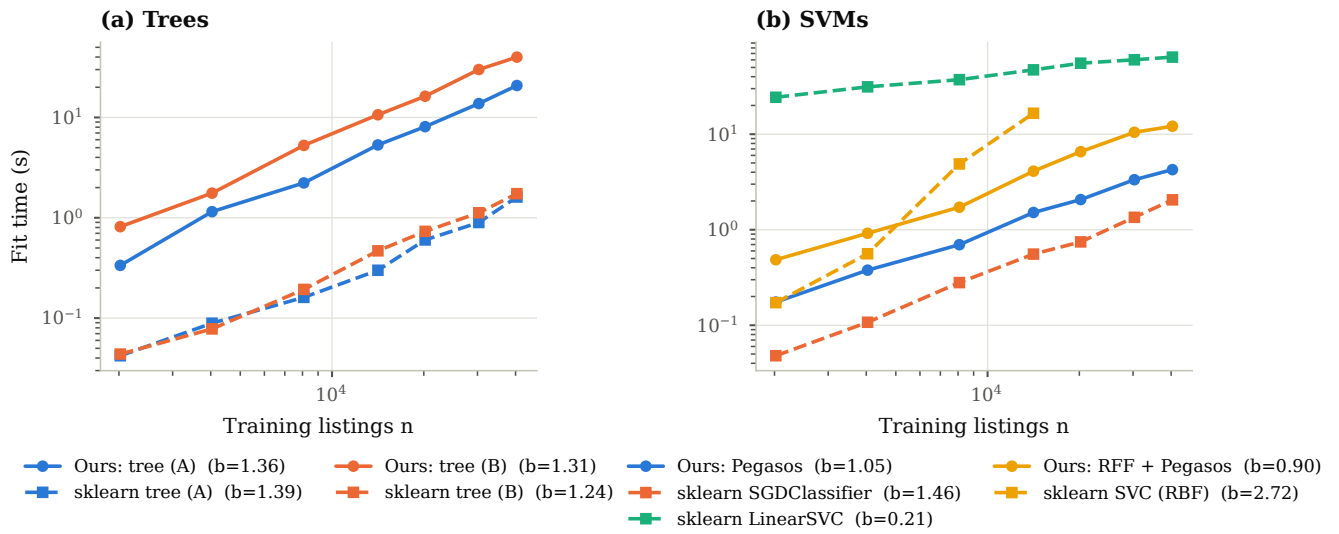


Figure 15. Fit time vs. number of training listings (log-log), trees (a) and SVMs (b); b is the fitted exponent of $t \approx an^b$. Solid: ours; dashed: scikit-learn. LIBLINEAR's time depends on how many dual iterations it needs at each C , hence its uneven curve.